

Age, the film's run and projected box office results, true cinematic to the success or failure, specifically related to the production company, its movie's budget, or other variables play a significant role in determining a movie's success. By identifying a data analysis using Python, the project aims to provide valuable insights that can help movie production companies make more informed decisions about which movies to produce, which companies to partner with, and how to allocate resources to maximize the chances of success

```
import pandas as pd
```

```
# Load both datasets
movies_df = pd.read_csv('tmdb_5000_movies.csv')
credits_df = pd.read_csv('tmdb_5000_credits.csv')
import os
os.getcwd()
```

Out[6]: 'C:\Users\shubhangi singh\movie_success analysis'

```

In [37]: import pandas as pd

movies_df = pd.read_csv('C:\Users\shubhangi singh\movie_success analysis\tmdb_5000_movies.csv')
```

```

In [11]: # Flatten all genres and count the frequency
         from collections import Counter

         # Flatten the genres into a single list
         all_genres = [genre for sublist in movie_data['genres'] for genre in sublist]
         genre_counts = Counter(all_genres)

```

```
genre_df = pd.DataFrame(genre_counts.items(), columns=['Genre', 'Count'])
genre_df = genre_df.sort_values(by='Count', ascending=False)

# Preview the result
genre_df.head(10)
```

```
Out[11]:
```

	Genre	Count
2	*	72960
6		44408
5	:	24320
13	e	20000
9	.	19645

```

11      a 19497
12      m 18626
13      i 18234
10      n 17186
14      d 14672

In [13]: # Calculate ROI
movie_data['ROI']

# Preview the result
movie_data[['r', 'ROI']]

Out[13]:

0
1  Pirates of the Ca
2
3
4

```

```
[51]: from wordcloud import WordCloud
import matplotlib.pyplot as plt
import ast

# Convert 'keywords' to list using ast
movie_data['keywords_list'] = movie_data['keywords'].dropna().apply(lambda x: [d['name'] for d in ast.literal_eval(x)] if x != '')
```

```
threshold = movie_data['revenue'].quantile(0.80)
successful_movies = movie_data[movie_data['revenue'] >= threshold]

# Combine all keywords of successful movies into one string
successful_keywords = successful_movies['keywords'].dropna(1).apply(lambda x: ' '.join([d['form'] for d in ast.literal_eval(x)]).str.cat(sep=' '))

# Generate word cloud
wordcloud = WordCloud(width=800, height=400, background_color='black', colormap='plasma').generate(successful_keywords)

# Plot it
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top Keywords in Successful Movies (Top 20% by Revenue)')
plt.show()
```

Top Keywords in Successful Movies (Top 20% by Revenue)

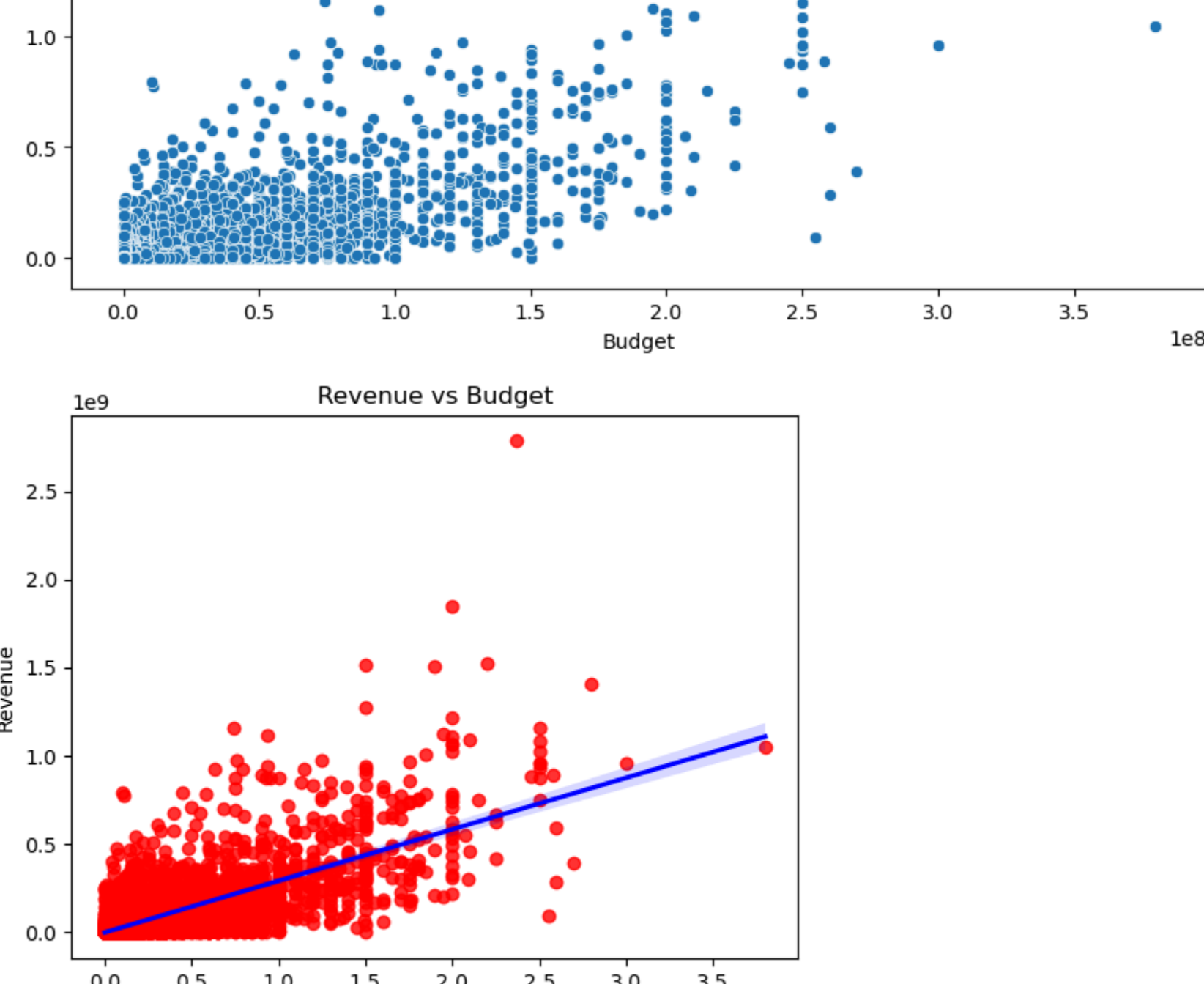


```
plt.figure(figsize=(10, 8))
plt.title('Correlation Between Movie Success Factors')
plt.show()
```



1e9 Budget vs Revenue

employees	revenue
1	1.2
2	1.5
3	1.3
4	1.5
5	1.9
6	1.5
7	2.7
8	1.4
9	1.4
10	1.4



	Budget	Actual
1. Total	100	100
2. Government	10	10
3. Private	90	90
4. Total	100	100

```

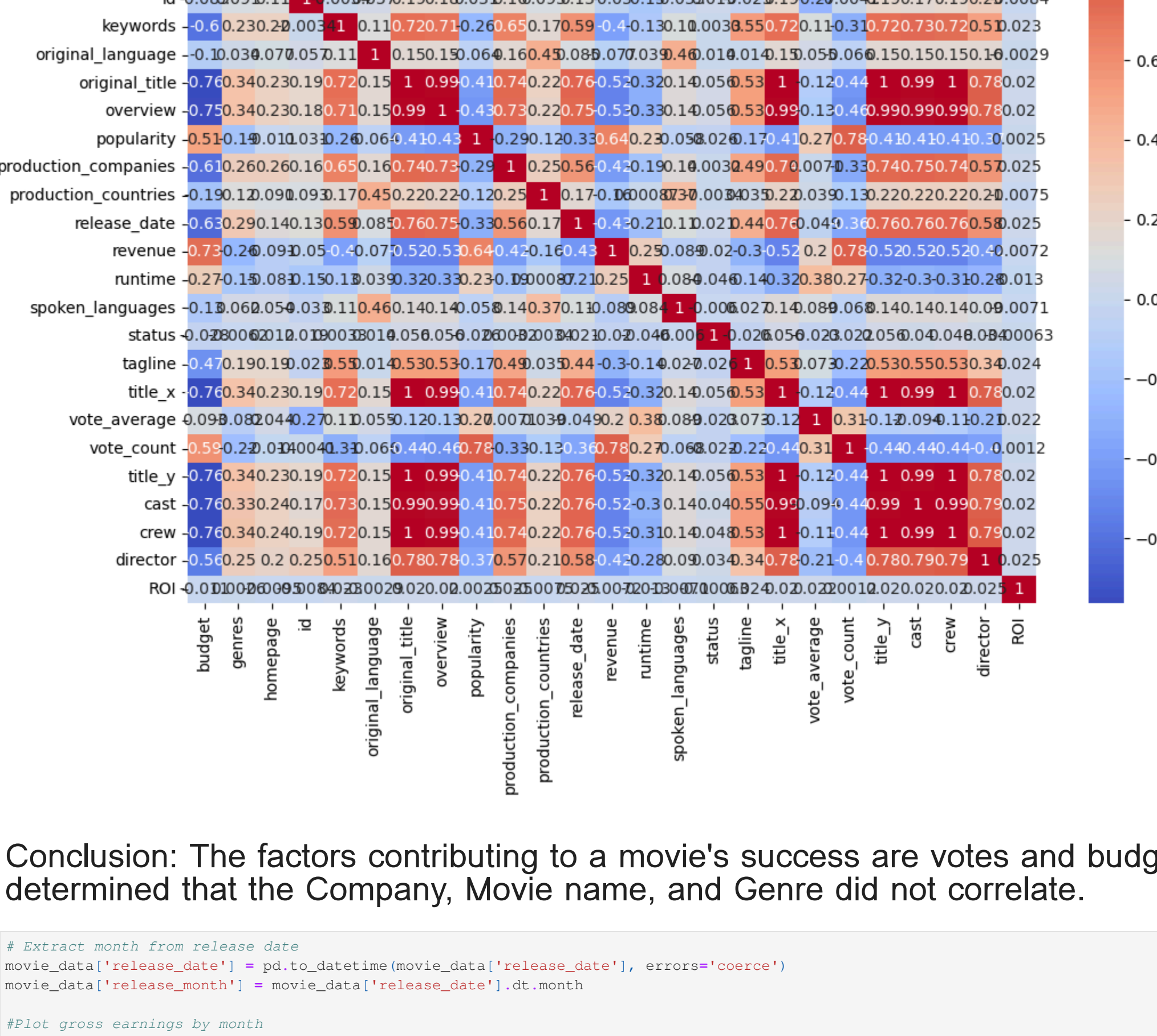
In [23]: import numpy as np

# Make a copy of your main DataFrame
df_numerized = movie_data.copy()

# Convert object (categorical) columns to numeric codes
for col in df_numerized.columns:
    if df_numerized[col].dtype == 'object':
        df_numerized[col] = pd.factorize(df_numerized[col])[0]
correlation_matrix = df_numerized.corr(method='pearson')

plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix for Movie Features')
plt.show()

```



Release Month	Number of releases
1	1
2	2
3	2.5
4	2.5
5	4.5
6	6
7	4
8	2.5
9	2.5
10	2.5
11	4.5
12	5

Conclusion: The best time of year to release

```

credits_df = pd.read_csv(r'C:\Users\shubhangi.singh\AppData\Local\Temp\ipykernel_16620\2462178525.py#41: UserWarning: set_tick_labels() is deprecated. Use plt.xticks() instead.
# Merge on 'id'
credits_df.rename(columns={'movie_id': 'id'}, inplace=True)
movie_data = movie_data.merge(credits_df, on='id')

# Simplify the 'genres' column (extract first genre)
def extract_genre(genre_str):
    try:
        genre_list = set literal_eval(genre_str)
        return genre_list[0]['name'] if genre_list else 'Unknown'
    except:
        return 'Unknown'

movie_data['genre'] = movie_data['genres'].apply(extract_genre)

# Group by genre and calculate sum and mean of revenue
grouped_df = movie_data.groupby('genre')[['revenue']].agg(['sum', 'mean']).sort_values(by='sum', ascending=False)

# Prepare data for plotting
genres = grouped_df.index.tolist()
sum_revenue = grouped_df['sum'].tolist()
mean_revenue = grouped_df['mean'].tolist()

# Plotting
fig, ax1 = plt.subplots(figsize=(14, 8)) # Larger figure

# Bar plot for total revenue
color = 'tab:blue'
ax1.bar(genres, sum_revenue, color=color, label='Total Revenue')
ax1.set_xlabel('genre')
ax1.set_ylabel('Total Revenue (USD)', color=color)
ax1.tick_params(axis='y', labelcolor=color)
ax1.set_title('Total and Average Revenue by Genre')
ax1.set_xticklabels(genres, rotation=45, ha='right')

# Second axis for average revenue
ax2 = ax1.twinx()
color = 'tab:orange'
ax2.bar(genres, mean_revenue, color=color, alpha=0.5, label='Average Revenue')
ax2.set_ylabel('Average Revenue (USD)', color=color)
ax2.tick_params(axis='y', labelcolor=color)

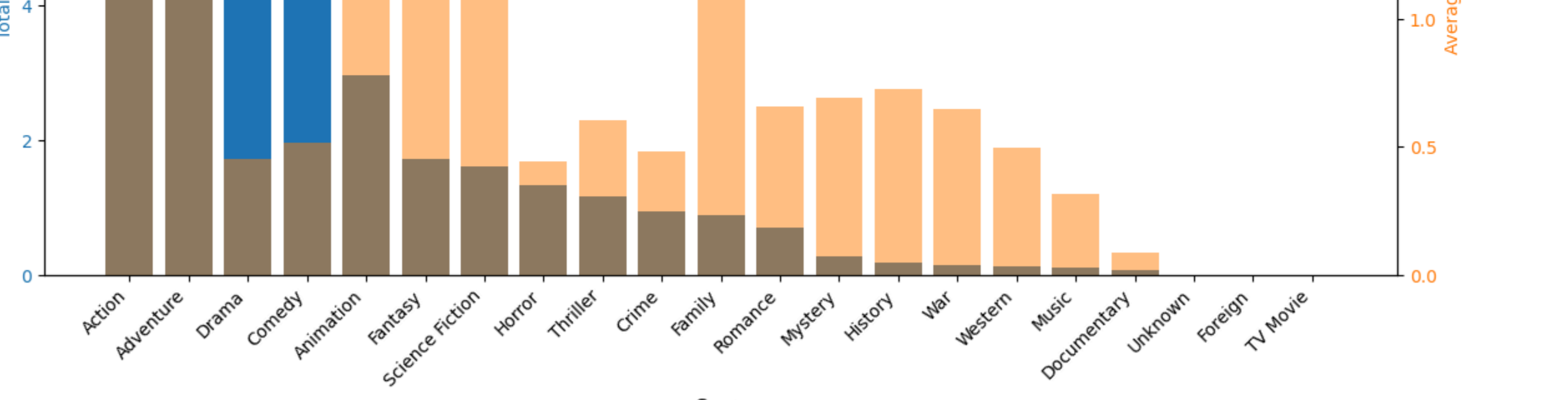
# Add legends
fig.legend(loc='upper right')

# Manual layout adjustment
plt.subplots_adjust(bottom=0.25, top=0.9)

plt.show()

```

Genre	Total Revenue (USD)	Average Revenue (USD)
Comedy	~8.5	~1.5
Drama	~7.5	~1.5
Action	~6.5	~1.5
Horror	~5.5	~1.5
Thriller	~4.5	~1.5
Science Fiction	~3.5	~1.5
Documentary	~2.5	~1.5
Animation	~1.5	~1.5
Other	~1.5	~1.5



Conclusion: The Action and animation genre had the highest level of success across all movies, whereas the Family genre had the highest average success rate.

Overall, my Python data analysis project provided valuable insights into the factors contributing to a movie's success.